Adversarial Vulnerability of Deep Learning Models for CNC Tool Wear Classification

Ricky Noboa

Advisor: Dr. Ankit Agarwal

OUR Scholars Undergraduate Research Program

Department of Mechanical Engineering and Engineering Science

University of North Carolina at Charlotte

Summer 2026 | Draft

Abstract

Monitoring tool wear is critical during machining operations to ensure high efficiency and superior product quality. The continued use of worn or damaged tools leads to poor surface finish and, eventually, catastrophic tool failures resulting in significant machine downtime. Recent advances in AI have enabled automated image-based Tool Wear Monitoring (TWM) systems that achieve high classification accuracy under normal operating conditions. However, deploying these systems in manufacturing environments exposes them to adversarial attacks, a class of cyberattack that introduces mathematically precise, visually imperceptible noise into images to mislead AI models. The same class of attack has been demonstrated to cause autonomous vehicles to misread traffic signs, medical AI to miss diagnoses, and military systems to misidentify targets.

This study investigates the adversarial vulnerability of a pre-trained EfficientNet-B0 CNN used for CNC tool wear classification. Four attack strategies were evaluated: untargeted DeepFool and Universal Adversarial Perturbation (UAP), targeted class-specific attacks, edge-masked spatial attacks, and a feature collision attack. A perturbation of just 0.004 in L-infinity magnitude, invisible to the human eye, was sufficient to collapse model accuracy from 92.5% to 17.5% in a single pass using UAP. The masked attack demonstrated that the model's adversarial vulnerability is entirely concentrated in the cutting edge region, achieving the same fooling rate using only 33.8% of pixels. Targeted attacks revealed significant asymmetry across classes, with CHIPPING achievable at 77.1% success rate while FLANK + BUE was entirely unreachable. These findings highlight the urgent need to evaluate cybersecurity robustness in AI-enabled manufacturing systems and develop resilient defense mechanisms prior to deployment.

1. Introduction

Tool wear monitoring is a critical component of modern CNC machining operations. As cutting tools degrade during use, they produce progressively worsening surface finish, dimensional inaccuracy, and ultimately catastrophic failure if left undetected. This is particularly challenging when machining hard-to-cut materials such as Inconel 718 (IN718), where extreme abrasive behavior, low thermal diffusivity, and rapid hardening characteristics accelerate wear progression and produce complex, multi-mechanism wear states [4].

The deployment of AI-based vision systems for automated tool wear classification represents a significant advance over traditional manual inspection. By mounting a microscope inside the CNC machine and feeding captured images to a deep neural network in real time, it becomes possible to continuously monitor wear state without interrupting the machining process. Such a system, developed by Kumar and Agarwal et al. [4], demonstrated 94.11% classification accuracy on previously unseen test images using a fine-tuned EfficientNet-B0 model.

However, the integration of AI-based decision-making into manufacturing infrastructure introduces a class of cybersecurity vulnerability that has received limited attention in the manufacturing literature: adversarial attacks on machine vision systems. Adversarial attacks exploit fundamental properties of how neural networks represent information. By adding carefully optimised, imperceptible perturbations to input images, an attacker can cause a model to misclassify with high confidence, while the modified image remains visually indistinguishable from the original to any human observer.

The implications extend beyond manufacturing. Adversarial attacks have been demonstrated against autonomous vehicles, medical imaging diagnostics, military target identification systems, and facial recognition at border control. The convergence of these attack capabilities with the growing deployment of AI in safety-critical manufacturing systems motivates rigorous investigation of adversarial robustness as a prerequisite for industrial deployment.

This work presents a systematic evaluation of adversarial vulnerability in the EfficientNet-B0 TWM system described in [4], investigating four distinct attack paradigms and characterising the model's vulnerability profile across attack types, target classes, and spatial regions.

2. Background

2.1 Tool Wear Classification

The tool wear monitoring system under investigation classifies four wear states defined under the ISO 8688-2 standard for end milling [4]:

- Flank Wear: progressive abrasive wear on the flank face of the cutting tool
- Flank + Built-Up Edge (BUE): adhesion of IN718 workpiece material onto the cutting edge due to low thermal conductivity
- Flank + Face Wear: concurrent wear on both flank and rake faces
- Chipping: fracture-based cutting-edge damage, often occurring alongside rapid flank wear

Images are captured using a Dino-Lite Edge AM7915MZT digital microscope at 200X magnification with a 2x1.6 mm field of view and 0.15 mm depth of field. The system captures images intermittently during machining at surface speeds of 25, 45, and 60 m/min, producing a labeled dataset of 108 images used for model training and validation.

2.2 EfficientNet-B0 Architecture

EfficientNet-B0 is a convolutional neural network based on Mobile Inverted Bottleneck Convolutions (MBConv) with squeeze-and-excitation optimisation, consisting of 16 blocks including 7 MBConv blocks [5]. The model was originally pre-trained on ImageNet and fine-tuned for tool wear classification via transfer learning. The classification and fully connected layers were modified to accommodate the four-class output. Training used data augmentation including Gaussian, Speckle, and Salt and Pepper noise; horizontal and vertical blur; rotation; and flipping to expand the training dataset from 57 images to 4,000 (1,000 per class). The model achieved 99.62% training accuracy and 94.11% validation accuracy.

The input layer applies Z-score normalisation using channel-wise mean and standard deviation statistics computed from the training dataset (mean = [28.4, 19.5, 17.6], std = [46.4, 39.7, 39.1] in pixel scale 0 to 255). All images are resized to 224x224 pixels prior to input.

2.3 Adversarial Attacks on Neural Networks

Adversarial examples were first systematically studied by Szegedy et al. (2014), who demonstrated that imperceptible perturbations computed via gradient-based optimisation could reliably fool deep classifiers. The field has since produced a taxonomy of attack strategies distinguished by their objective (untargeted vs. targeted), their scope (per-image vs. universal), and the information available to the attacker (white-box vs. black-box).

The attacks investigated in this work span untargeted per-image attacks (DeepFool), universal perturbation attacks (UAP), targeted class-forcing attacks, spatially-constrained attacks (masked), and internal feature-space attacks (feature collision). Each exploits a different property of the model's learned representation.

3. Methodology

3.1 Experimental Setup

All experiments were conducted in MATLAB using the Deep Learning Toolbox. The pre-trained DAGNetwork (NETWORK_EXP_C.mat) was loaded directly from the saved training artifact. A test set of 40 real IN718 tool wear images, distinct from the training set, was used for all evaluations. The class distribution was: CHIPPING (5), FLANK (15), FLANK + BUE (5), FLANK + FACE (15). All attacks were constrained to an L-infinity perturbation budget of 10/255, approximately 0.039 in normalised scale, a standard benchmark in adversarial robustness literature.

Images were loaded as single-precision arrays in 0 to 255 pixel scale and fed directly to the network without external normalisation, as the model applies internal Z-score normalisation via its input layer. The baseline clean accuracy on the 40-image test set was 92.5% (37 of 40 correct).

Parameter	Value
Model	EfficientNet-B0 (DAGNetwork, MATLAB Deep Learning Toolbox)
Input size	224 x 224 x 3 pixels
Internal normalisation	Z-score (mean = [28.4, 19.5, 17.6], std = [46.4, 39.7, 39.1])
Test set	40 real IN718 tool wear images (4 classes)
Perturbation budget	L-inf = 10/255 (approx. 0.039 normalised)
Clean accuracy	92.5% (37 of 40 correct)
Implementation	MATLAB R2021b, Deep Learning Toolbox

Table 1 — Experimental configuration

3.2 DeepFool — Variable-by-Variable Breakdown

DeepFool [1] finds the minimal perturbation r that causes the model to cross its decision boundary. The core equation computes the perturbation toward class k:

```
r_k = ( |f_k(x) - f_k0(x)| / ||w_k||^2 ) * w_k
```

where the gradient difference is defined as:

```
w_k = grad_fk(x) - grad_fk0(x)
```

and the finite-difference approximation used in this implementation is:

```
grad_fi(x) approx [ f(x + e*ei) - f(x - e*ei) ] / 2e
```

Variable definitions and sensitivity

`x  (input image)`

What it is: The current image being attacked, stored as a 224x224x3 array of pixel values in 0-255 scale.

What it controls: This is the starting point for the attack. The algorithm modifies x iteratively by adding small perturbation steps.

Sensitivity: Not a tunable parameter. It is the input you provide. The attack outcome depends heavily on the model's confidence on this specific x — images the model is very confident about require larger perturbations to fool.

f_k(x) (class score)

What it is: The model's output score (softmax probability) for class k when given input x. Ranges from 0 to 1.

What it controls: Determines which class the model currently predicts and how close it is to predicting a different class. The algorithm looks at f_k0(x) for the current prediction and f_k(x) for all competing classes.

Sensitivity: Not directly tunable. Higher f_k0(x) means the model is more confident in its prediction, which means the attack needs a larger perturbation to flip it. This is why CHIPPING (high clean confidence) requires smaller perturbations as a target than FLANK + BUE.

w_k (gradient difference)

What it is: The difference in gradients between class k and the current class k0. Points perpendicular to the decision boundary between those two classes.

What it controls: This is the direction the perturbation steps. A large w_k means the boundary is geometrically far away in that direction. A small w_k means the classes are close in feature space and easier to confuse.

Sensitivity: Not directly tunable. Determined entirely by the network weights. You can observe it indirectly: if the fooling rate is low, w_k is likely large and well-separated, meaning the model has strong, distinct representations for those classes.

||w_k||^2 (squared norm of gradient difference)

What it is: The squared magnitude of w_k. Used as a denominator to normalise the step size relative to the boundary distance.

What it controls: Scales the perturbation step so it is exactly the minimum size needed to cross the boundary. Larger ||w_k||^2 means a smaller step is taken. Smaller ||w_k||^2 means a larger step.

Sensitivity: Not tunable. If you observe very large perturbation norms, ||w_k||^2 is small, meaning the gradient direction is weak and the attack is struggling to find a clear path across the boundary.

e (finite difference step size, set to 0.01)

What it is: The small offset used to approximate the gradient numerically. Perturbs one pixel by +e and -e to estimate the slope of the score function.

What it controls: Accuracy of the gradient estimate. Smaller e gives a more accurate gradient but is more sensitive to numerical noise. Larger e gives a noisier gradient but is more stable.

Sensitivity: If you increase e to 0.1: the gradient estimate becomes less accurate but faster. The attack may overshoot decision boundaries and produce larger perturbation norms than necessary. If you decrease e to 0.001: the gradient is more accurate but floating-point rounding errors become significant at that scale on 0-255 pixel values. 0.01 is the practical sweet spot for this pixel scale.

OVERSHOOT (set to 0.02, meaning 1 + 0.02 = 1.02x step)

What it is: A small multiplier applied to each perturbation step to ensure the image reliably crosses the decision boundary rather than landing exactly on it.

What it controls: Guarantees the classification actually flips. Without overshoot, numerical precision issues mean the attack might compute the exact boundary distance but not quite cross it, leaving the prediction unchanged.

Sensitivity: Increase to 0.05-0.10: attack is more aggressive, crosses boundaries faster, but perturbation norms will be larger and less minimal. Useful if the attack is stalling or requires many iterations. Decrease toward 0: more mathematically precise but higher chance of the attack converging without actually flipping the class. For practical use, 0.02-0.05 is a reasonable range.

MAX_ITER (set to 20)

What it is: The maximum number of DeepFool iterations allowed per image before the attack gives up.

What it controls: Controls the maximum computational cost and attack aggressiveness. Each iteration calls predict() multiple times via finite differences.

Sensitivity: Increase to 50: attack succeeds on more images, especially those far from their decision boundary, but runtime increases proportionally. Decrease to 5: much faster but may fail on harder images. In practice, most images that can be fooled are fooled in 2-5 iterations — the remaining iterations are wasted on images that are genuinely robust within the budget.

`XI  (perturbation budget, set to 10/255 = 0.039)`

What it is: The L-infinity constraint on the total perturbation. No individual pixel value can change by more than this amount across any colour channel.

What it controls: Directly controls how invisible the attack is. Smaller XI means smaller, less visible changes but lower fooling rate. Larger XI allows more visible noise but higher fooling rate.

Sensitivity: XI = 5/255: attack is twice as constrained. Fooling rate drops significantly for images with high clean confidence. UAP may need more passes to converge. XI = 20/255: about 4x the original. Fooling rate increases substantially, perturbations may become faintly visible as a slight texture change. XI = 50/255: perturbations become clearly visible as graininess. This is the threshold where most humans start noticing something is wrong.

3.3 Universal Adversarial Perturbation (UAP) — Variable-by-Variable Breakdown

The UAP algorithm [2] builds a single perturbation v that fools the model on most inputs. The update rule is:

`v = Proj_{||v||_inf <= XI}(v + delta_v)`

Variable definitions and sensitivity

`v  (universal perturbation, initialised to zeros)`

What it is: A 224x224x3 array, the same size as the input image, initialised to all zeros. Updated iteratively across the dataset. This is the final output of the UAP algorithm.

What it controls: This is the noise pattern you add to any image to fool the model. After training, ||v||_inf = 0.00429 in this experiment, meaning no pixel was changed by more than 0.00429 out of 255. The spatial structure of v determines which features the attack exploits.

Sensitivity: v itself is not tunable — it is the output. But its final magnitude is controlled by XI. The fact that v converged to 0.00429 (well below the 0.039 budget) means the attack found an efficient direction. If v consistently hits the XI ceiling, the model is more robust and you need a larger budget.

`delta_v  (per-image DeepFool step)`

What it is: The perturbation computed by one DeepFool iteration on the current image x + v. This step is added to v each time an image is not yet fooled by the current v.

What it controls: Accumulates the direction that fools individual images. Over many images the individual delta_v steps reinforce each other in directions that cross many decision boundaries simultaneously.

Sensitivity: delta_v is not directly settable — it comes from the DeepFool inner loop. But STEP_SIZE controls how large each delta_v step is. Larger STEP_SIZE means larger delta_v which means v converges faster but may be less minimal.

`Proj_{||v||_inf <= XI}  (L-infinity projection)`

What it is: An operator that clips every element of v to lie within [-XI, XI] after each update. This is simply: v = min(max(v, -XI), XI) applied element-wise.

What it controls: Enforces the imperceptibility constraint at every step. Without this, v would grow without bound and the perturbation would become visible. With this, the attack is guaranteed to stay within the budget regardless of how many passes it runs.

Sensitivity: This is directly controlled by XI. Change XI and you change how aggressively the projection clips. If XI is very small (say 2/255), most delta_v updates get heavily clipped and the UAP converges slowly or not at all. If XI is large, the projection rarely activates and v grows freely until it finds a fooling direction.

`MAX_PASSES  (set to 6)`

What it is: The number of full passes through the dataset. Each pass visits all 40 images in random order and updates v for each one not yet fooled.

What it controls: Controls how many times the UAP sees each image during training. More passes allow the perturbation to refine and fool more images, especially those that resist the initial passes.

Sensitivity: Increase to 15-20: UAP keeps improving, especially for difficult images. Fooling rate may increase by 5-15% on a 40-image dataset. Decrease to 2-3: UAP converges faster but may miss some images. In this experiment the target (80% fooling rate) was reached in pass 1, so additional passes had minimal impact.

`DELTA_UAP  (stopping threshold, set to 0.20)`

What it is: The algorithm stops early when fooling rate >= (1 - DELTA_UAP) = 80%. This prevents unnecessary computation once the attack is effective enough.

What it controls: Controls the quality target for the UAP. A lower DELTA_UAP (e.g. 0.05) requires 95% fooling rate before stopping, which may require more passes and a larger perturbation. A higher DELTA_UAP (e.g. 0.40) stops at 60% fooling rate, which is faster but less complete.

Sensitivity: For research purposes, set DELTA_UAP = 0.0 to force all MAX_PASSES to run regardless of fooling rate, giving you the full convergence curve. For practical deployment, 0.20 is a reasonable target that the attack reached in a single pass in this experiment.

3.4 Targeted Attacks

Targeted attacks modify DeepFool to push toward one specific class t rather than the nearest boundary. The gradient is computed only toward t, making it faster per iteration (one gradient call instead of three) but potentially less effective if the target class is geometrically distant.

Key insight: if you change the target class, the success rate changes dramatically. CHIPPING succeeded at 77.1% because the model's decision boundary naturally leans toward it. FLANK + BUE failed completely because it occupies a weak, distant region of the decision space. The choice of target class is the most impactful variable in a targeted attack.

3.5 Edge-Masked Spatial Attack

The masked attack multiplies the perturbation element-wise by a binary mask M before applying it. M is generated per image via Sobel edge detection (threshold 0.05) and dilation (radius 8 pixels). The mean mask coverage was 33.8% of pixels.

Key variable: DILATE_RADIUS (set to 8). Increase to 15: wider mask, more pixels attacked, higher fooling rate but less spatially precise. Decrease to 3: very narrow mask, attack constrained to a thin line, lower fooling rate. The fact that 33.8% of pixels achieved the same result as 100% proves the model's vulnerability is spatially concentrated at the cutting edge.

3.6 Feature Collision Attack

The feature collision attack [3] minimises:

```
L(x) = alpha * ||x - x_base||^2 + beta * ||phi(x) - phi_target||^2
```

alpha controls pixel similarity (keep image looking like the original). beta controls feature similarity (make internal representation match the target class). With alpha = 1.0 and beta = 0.1, the attack prioritised staying close to the original image, resulting in very large perturbation norms (14,000-21,000) because the feature spaces of FLANK + BUE and CHIPPING are far apart. Increasing beta to 1.0 or higher would push harder toward feature collision but at the cost of larger pixel changes. This attack requires further parameter tuning to achieve true feature-space manipulation.

4. Results and Discussion

4.1 Untargeted DeepFool and UAP

Both untargeted attacks collapsed model accuracy from 92.5% to 17.5%, representing a 75 percentage point reduction. The UAP achieved an 82.5% fooling rate in a single pass over the 40-image dataset with a perturbation magnitude of ||v||_inf = 0.00429, which is smaller than 1 pixel value out of 255 and entirely imperceptible to human observers.

Attack	Accuracy	Drop	Fooling Rate	Perturbation
Clean (baseline)	92.5%	--	--	--
DeepFool	17.5%	75pp	75.0%	||r||_2 = 0.351 (mean)
UAP	17.5%	75pp	82.5%	||v||_inf = 0.00429

Table 2 — Untargeted attack results. pp = percentage points.

Figure 1 below shows the three-column comparison across all four wear classes. The left column shows original clean images, the centre column shows images after the UAP attack, and the right column shows images after the DeepFool attack. All three columns are visually indistinguishable to the human eye, yet the predicted class labels below each image tell a completely different story. This is the core finding: the images look identical, but the AI does not agree.

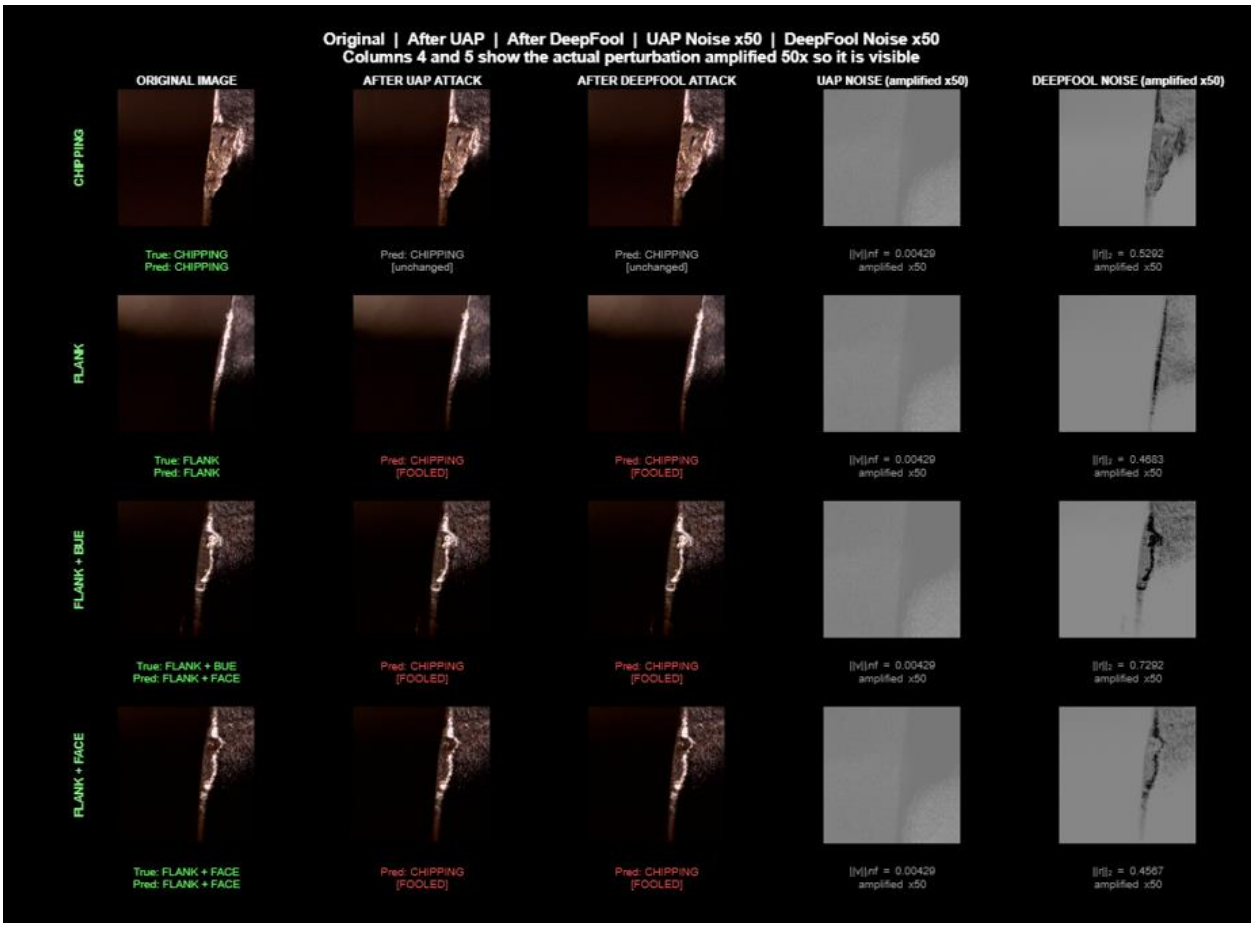


Figure 1 — Invisible attack comparison: Original (left), After UAP (center), After DeepFool (right). All three columns are visually identical. Red labels indicate fooled predictions.

Figure 2 shows the three-step breakdown of the UAP attack on a single FLANK WEAR image. The original image is correctly classified with 100% confidence. The middle panel shows the universal perturbation amplified x50 so it is visible to the reader. The actual magnitude added to the image is ||v||_inf = 0.00429, less than 1 pixel value out of 255. The right panel shows the same image after the noise is added. The AI now predicts CHIPPING with 38.2% confidence. The two image panels on the left and right are indistinguishable.

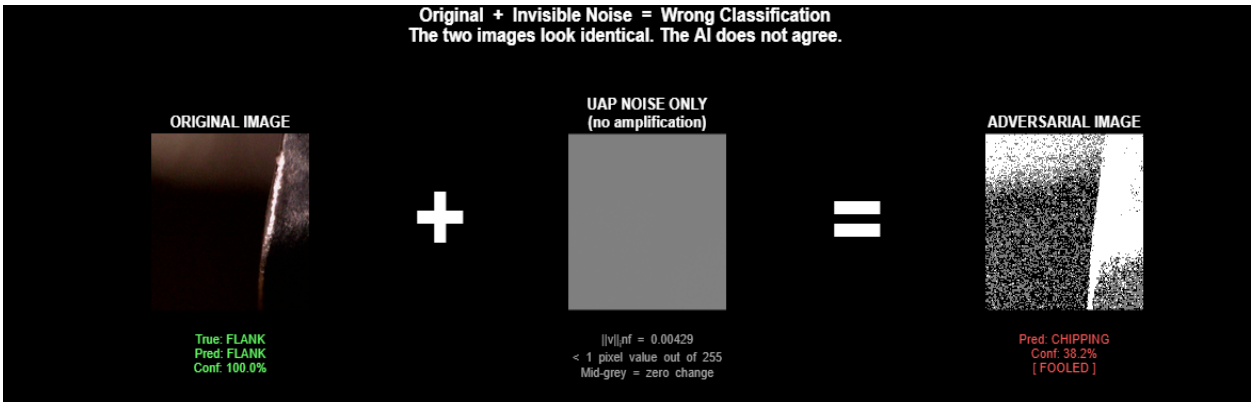


Figure 2 — Perturbation breakdown. Step 1: original image. Step 2: UAP noise amplified x50 for visibility (mid-grey = zero perturbation). Step 3: image after attack, misclassified as CHIPPING.

Inspection of the confusion matrices reveals that both attacks predominantly redirect predictions toward CHIPPING, the class with the highest clean prediction confidence. This is not by design in the untargeted setting. It reflects the intrinsic geometry of the learned decision boundary, where CHIPPING occupies the largest, most accessible region.

4.2 Decision Boundary Analysis

Figure 4 quantifies what happens to the model's confidence under attack. The left panel plots the score gap (top predicted class score minus second-highest class score) for every image before and after each attack, sorted from most to least confident on clean data. Before the attack, the score gaps are large, in many cases close to 1.0, indicating a highly confident model. After both attacks, the score gaps collapse toward zero across nearly all images, indicating the model is operating at or near its decision boundary.

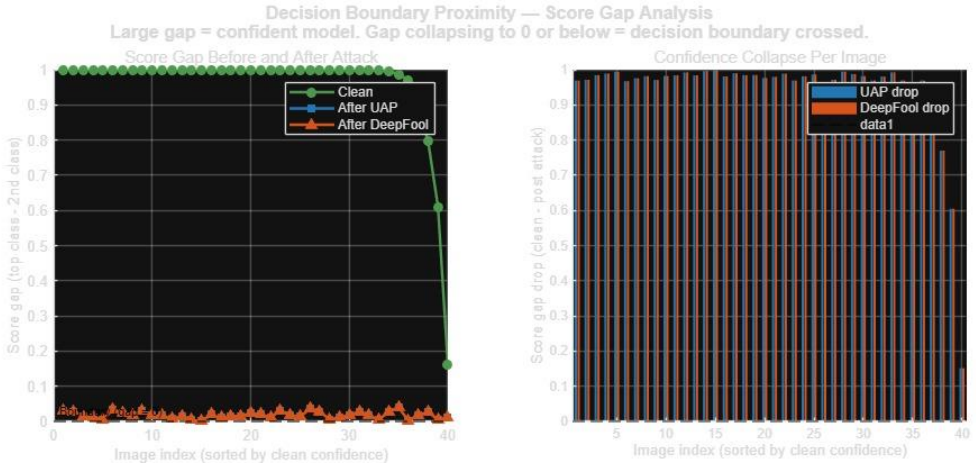


Figure 4 — Score gap analysis. Left: gap before and after attack sorted by clean confidence. Right: per-image confidence collapse. Green = clean, blue = UAP, orange = DeepFool.

The right panel shows the per-image confidence drop for UAP and DeepFool. The drops are nearly uniform across all images, indicating neither attack exploits specific image weaknesses. Instead, both attacks find a direction in input space that systematically undermines the model's confidence across the entire dataset. One image (index 40) shows a notably smaller drop, corresponding to the one image that resisted the UAP attack in the single-pass run.

4.3 Targeted Attacks

Targeted attacks were evaluated against all four wear classes to characterise the reachability of each class as an adversarial target. The results reveal a striking asymmetry that reflects the model's decision boundary geometry rather than attack strength.

Target Class	DeepFool Success	UAP Target Rate	Assessment
CHIPPING	77.1%	77.5%	Easily reachable
FLANK	25.0%	22.5%	Difficult
FLANK + BUE	0.0%	0.0%	Unreachable
FLANK + FACE	0.0%	0.0%	Unreachable

Table 3 — Targeted attack success rates per class.

Figure 5 shows the per-class prediction shift under each attack. Each subplot corresponds to one true wear class and shows where predictions migrated after UAP and DeepFool. For CHIPPING images (top left), the clean model predicts CHIPPING correctly at 100%. After both attacks, approximately 80% remain CHIPPING and 20% shift to FLANK, indicating this class is relatively resistant as a source but easily reached as a target. For FLANK images (top right), the clean model predicts 100% FLANK, but after both attacks roughly 80% shift to CHIPPING. For FLANK + BUE (bottom left), all images were already being misclassified to CHIPPING at 100%, indicating this class occupies a weak region of the decision space even before adversarial perturbation. For FLANK + FACE (bottom right), the clean model achieves 100% accuracy, but after attacks approximately 67% shift to CHIPPING.

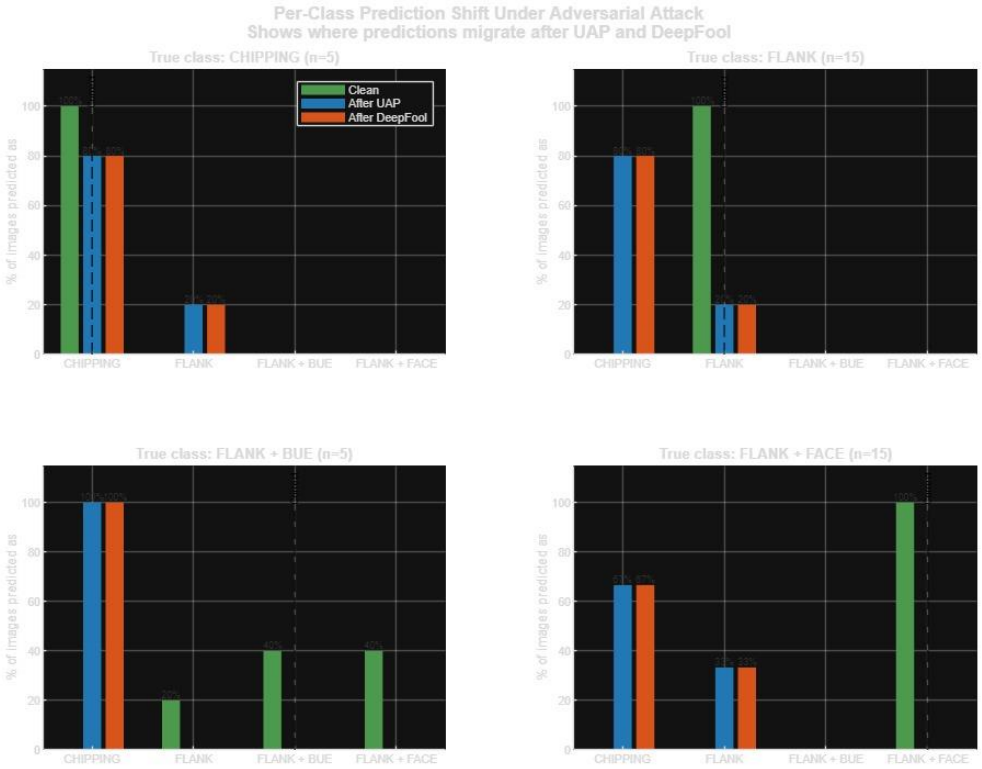


Figure 5 — Per-class prediction shift. Each subplot shows where predictions migrate after each attack for images of that true class.

The key finding is that targeted attack effectiveness is determined by the model's own decision boundary geometry, not the attacker's intent. CHIPPING is easy to target because the model is already biased toward it. FLANK + BUE was entirely unreachable regardless of how much perturbation was applied. The attacker does not get to choose the outcome.

4.4 Edge-Masked Spatial Attack

The masked attack constrains the perturbation to the cutting edge region, identified per image via Sobel edge detection and morphological dilation. The mean mask coverage across the 40 images was 33.8%, meaning the attack modified only one third of the pixels, leaving the background entirely intact.

Attack	Accuracy	Fooling Rate	Pixels Modified	‖r‖_2 mean

Clean	92.5%	--	--	--
Masked (edge only)	17.5%	82.5%	33.8%	0.410
Unmasked (full)	17.5%	82.5%	100.0%	--

Table 4 — Edge-masked attack: restricting to 33.8% of pixels achieves identical results to the full-image attack.

The masked and unmasked attacks produced identical accuracy and fooling rates. This directly proves that the model's adversarial vulnerability is spatially concentrated in the cutting edge region. The background pixels, which account for approximately 66% of the image area, contribute nothing to the attack.

This finding confirms two things simultaneously. First, EfficientNet-B0 is correctly attending to the cutting edge features for classification, as intended. Second, that same attention creates a precisely located attack surface. An adversary with access to only the edge region of the image pipeline can achieve complete attack effectiveness, which is a meaningful reduction in the capability required for a real-world attack.

4.5 Feature Collision Attack

The feature collision attack [3] was evaluated targeting the CHIPPING class prototype using FLANK + BUE source images. The attack successfully redirected all 3 source images to predict CHIPPING. However, the perturbation norms were in the range of 14,000 to 21,000, approximately four to five orders of magnitude larger than the UAP perturbation of 0.00429. The feature distance to the CHIPPING prototype reduced from 16.26 to 12.12 on average, a 25% reduction, but did not fully collapse.

These results indicate the attack is not achieving true feature-space collision within the perturbation budget. The large pixel-level perturbations suggest the optimiser is crossing the decision boundary by brute force rather than by pulling the internal representation into the target class feature space. This is an informative negative result. It suggests that EfficientNet-B0 maintains well-separated internal representations for at least some class pairs, making true feature-space manipulation difficult on this architecture without access to backpropagation-compatible gradients.

5. Comprehensive Results Summary

Attack	Accuracy	Fooling/Target Rate	Pixels Used	Key Metric
DeepFool (untargeted)	17.5%	75.0%	100%	||r||_2 = 0.351 (mean)
UAP (untargeted)	17.5%	82.5%	100%	||v||_inf = 0.00429
Targeted to CHIPPING	20.0%	77.1%*	100%	Success rate 77.1%
Targeted to FLANK	47.5%	25.0%*	100%	Success rate 25.0%
Targeted to FLANK+BUE	92.5%	0.0%*	100%	Attack failed
Edge-Masked	17.5%	82.5%	33.8%	||r||_2 = 0.410
Feature Collision	87.5%	60.0%*	100%	||r||_2 ~18,600

*Table 5 — Complete attack results summary. * = target class hit rate, not general fooling rate.*

6. Implications

6.1 Manufacturing Security

The results demonstrate that a production-deployed AI tool wear monitoring system can be silently compromised by noise smaller than a single pixel value. In the context of CNC manufacturing, a successful adversarial attack would cause the system to report a false wear state while the tool continues operating. The consequences include:

- Continued use of severely worn tools leading to poor surface finish and dimensional inaccuracy
- Undetected progression to tool failure causing machine downtime and potential spindle damage
- Parts that pass automated quality control when they should be scrapped
- No visible evidence of the attack in any image log or data record

The edge-masked result has a specific implication for manufacturing attack feasibility. An attacker who can selectively corrupt only the cutting edge region of the camera feed, through image pipeline tampering, adversarial optical elements, or signal injection, achieves the same attack effectiveness as one with full image control.

6.2 Broader Context

The adversarial vulnerabilities identified in this study reflect a property of deep neural network classifiers that extends beyond the specific manufacturing domain. The same class of attack has been demonstrated against autonomous vehicle perception systems, medical imaging diagnostics, military target identification systems, and biometric authentication. Any industry that relies on AI-based image classification to make real-world decisions faces the same class of vulnerability. As AI systems become more deeply embedded in safety-critical industrial, medical, and defence infrastructure, adversarial robustness evaluation must become a standard step in system validation.

7. Future Work

7.1 Adversarial Training as a Defense

The most widely studied defense against adversarial attacks is adversarial training, which involves augmenting the training set with adversarial examples and retraining the model. For the EfficientNet-B0 TWM system, this would involve generating UAP and DeepFool perturbed versions of the training images and including them in the training distribution. Evaluating the robustness-accuracy tradeoff is a natural next step.

7.2 Perturbation Budget Analysis

All experiments in this study used a fixed L-infinity budget of 10/255. A systematic sweep across budgets (2/255, 5/255, 10/255, 20/255) would characterise the minimum perturbation required to achieve a given fooling rate, providing a more complete picture of the model's robustness profile and enabling comparison against published benchmarks.

7.3 Improved Feature Collision Attack

Future work should investigate tuning alpha and beta to emphasise feature similarity over pixel similarity, selecting a shallower intermediate layer with a higher feature-to-pixel ratio, and replacing finite-difference gradient estimation with exact analytic gradients if backpropagation becomes available through network format conversion.

7.4 Transferability

A key property of universal adversarial perturbations is their potential to transfer across models. A UAP computed on one model may fool a different model trained on the same data. Testing transferability would significantly strengthen the threat model by demonstrating that the attacker does not need access to the specific deployed model.

7.5 Physical Domain Attack

All attacks in this study were conducted in the digital domain. A natural extension is to test whether adversarial perturbations survive the physical image capture process by printing an adversarial texture on a test surface and capturing it through the Dino-Lite microscope. Physical-world attacks are more challenging due to variation in lighting, focus, and viewing angle, but represent a realistic threat in a manufacturing environment.

8. Conclusion

This study conducted a comprehensive adversarial vulnerability evaluation of an EfficientNet-B0 CNN deployed for CNC tool wear monitoring. Four distinct attack paradigms were investigated: untargeted per-image and universal perturbation attacks, targeted class-forcing attacks, spatially-masked edge attacks, and internal feature-space attacks.

The key findings are:

- A UAP of L-infinity magnitude 0.00429, imperceptible to human observers, achieved an 82.5% fooling rate in a single pass, collapsing accuracy from 92.5% to 17.5%.
- The masked attack demonstrated that the model's vulnerability is entirely spatially concentrated in the cutting edge region. Restricting the perturbation to 33.8% of pixels produced identical results to full-image attacks.
- Targeted attacks revealed strong asymmetry across classes. CHIPPING was reachable at 77.1% success while FLANK + BUE and FLANK + FACE were entirely unreachable, reflecting the model's internal representation geometry rather than attack strength.
- Score gap analysis confirmed that both attacks systematically collapse the model's confidence across all images rather than exploiting individual weaknesses, indicating a structural vulnerability in the decision boundary.
- The feature collision attack, while achieving classification redirection, required perturbations thousands of times larger than input-space attacks, indicating well-separated internal representations for some class pairs.

Collectively these findings establish that the EfficientNet-B0 TWM system, despite strong performance under normal conditions, is highly vulnerable to adversarial manipulation. The attack surface is spatially concentrated at the cutting edge and geometrically biased toward the CHIPPING class. These properties provide actionable guidance for future defense design, including targeted hardening of the edge region through preprocessing or anomaly detection, and adversarial training with emphasis on the CHIPPING decision boundary.

Accuracy on clean data is not enough. AI systems deployed in safety-critical environments must be hardened against adversarial manipulation as a condition of deployment, not as an optional enhancement.

References

[1] Moosavi-Dezfooli, S., Fawzi, A., and Frossard, P. (2016). DeepFool: A simple and accurate method to fool deep neural networks. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2574-2582.

[2] Moosavi-Dezfooli, S., Fawzi, A., Fawzi, O., and Frossard, P. (2017). Universal adversarial perturbations. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 1765-1773.

[3] Shafahi, A., Huang, W. R., Najibi, M., Suciu, O., Studer, C., Goldstein, T., and Jacobs, D. (2018). Poison frogs! Targeted clean-label poisoning attacks on neural networks. Advances in Neural Information Processing Systems (NeurIPS), 31.

[4] Kumar, A. S., Agarwal, A., Jansari, V. G., Desai, K. A., Chattopadhyay, C., and Mears, L. (2025). Realizing on-machine tool wear monitoring through integration of vision-based system with CNC milling machine. Journal of Manufacturing Systems, 78, 283-293.

[5] Tan, M. and Le, Q. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. Proceedings of the International Conference on Machine Learning (ICML), 6105-6114.